Experiment: Analyze Synchronization Mechanisms in Parallel Programs

1. Objective

To implement and study common synchronization mechanisms used in shared-memory parallel programs - an unsynchronized (race) update, critical sections, atomic operations, barriers, and semaphores/locks - and to evaluate their correctness, execution time, and scalability as the number of threads increases.

2. Software Requirements

S.No.	Software / Tool	Purpose
1	VMware Workstation Pro	Virtualization software to host the Ubuntu guest OS
2	Ubuntu 22.04 LTS	Guest operating system used for compiling and running the programs
3	GCC Compiler (build-essential)	Compiles the C programs
4	OpenMP (libgomp, bundled with GCC)	Provides #pragma omp directives for parallel loops, critical, atomic, barrier
5	POSIX Threads (pthread) & Semaphore library	Used by the semaphore-based program
6	Ubuntu Terminal	To compile and execute all programs and record timing results

3. Theory

3.1 Why Synchronization is Needed

In a shared-memory parallel program, multiple threads can read and write the same variable at the same time. If two threads perform a read-modify-write sequence (such as counter++) on a shared variable without any coordination, their operations can interleave in a way that loses updates. This is called a race condition, and it produces results that depend unpredictably on thread scheduling. Synchronization mechanisms are language- and OS-level tools that control how threads access shared data so that the program's result stays correct regardless of how threads are scheduled.

3.2 Race Condition

A race condition occurs when the outcome of a program depends on the relative timing of multiple threads accessing shared data without protection. counter++ is not a single CPU instruction - it is a read, an increment, and a write. If two threads read the same old value before either writes back the incremented value, one update is silently lost. Race conditions are fast (no synchronization overhead) but produce incorrect and non-repeatable results.

3.3 Critical Section

A critical section (#pragma omp critical) is a block of code that only one thread may execute at a time; all other threads attempting to enter must wait. It guarantees correctness by serializing access to the shared resource, but because every thread must queue up for the same lock, it becomes a scalability bottleneck as

thread count grows - performance can even get worse as more threads are added, since more time is spent waiting than working.

3.4 Atomic Operation

An atomic operation (#pragma omp atomic) tells the compiler to perform a simple read-modify-write (like counter++) using an indivisible, hardware-supported instruction (e.g., a CPU compare-and-swap or lock-add instruction) rather than a general-purpose lock. Because atomics use lightweight hardware support instead of OS-level mutex machinery, they are correct and typically scale much better than critical sections for simple updates - though they only work for basic operations, not arbitrary code blocks.

3.5 Barrier

A barrier (#pragma omp barrier) is a synchronization point at which every thread must arrive before any thread is allowed to proceed further. It does not protect shared data directly; instead, it synchronizes the phases of a computation (e.g., ensuring all threads finish phase 1 before any thread starts phase 2). Barriers introduce waiting overhead equal to the time the slowest thread takes to reach the barrier.

3.6 Semaphore

A semaphore is a counter-based OS-level synchronization object. sem_wait() decrements the counter (blocking if it is already zero) and sem_post() increments it. A binary semaphore (initial value 1), as used in Program 5, behaves like a mutex lock, allowing only one thread into the protected region at a time. Semaphores are more general than mutexes - a counting semaphore can allow up to N threads into a region simultaneously - but that generality comes with moderate system-call overhead.

3.7 Correctness vs. Performance vs. Scalability

Three properties are used to judge a synchronization strategy: Correctness - does it always produce the right answer regardless of scheduling? Performance - how fast does it run for a given number of threads? Scalability - does performance improve (or at least not degrade) as more threads are added? An ideal mechanism is correct, fast, and scales well; in practice, stronger correctness guarantees usually cost more performance and scalability, which is exactly what this experiment measures.

4. Synchronization Mechanisms - Diagram

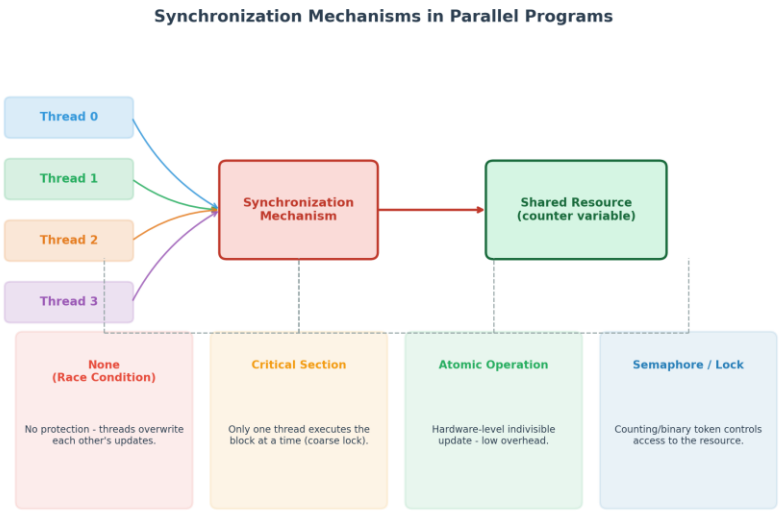


Fig. 1: Multiple threads accessing a shared resource through different synchronization mechanisms.

5. Installation Steps (From Scratch)

5.1 Install VMware Workstation Pro

1. Download VMware Workstation Pro from the official Broadcom/VMware website for your host OS.
2. Run the installer and follow the setup wizard, accepting the license agreement.
3. Choose the installation location and finish the installation (enter a license key if prompted, or use the free version for personal use).
4. Launch VMware Workstation Pro once installation completes.

5.2 Create a New Virtual Machine

5. Open VMware Workstation and click "Create a New Virtual Machine".
6. Select "Typical (recommended)" configuration and click Next.
7. Browse to the Ubuntu 22.04 LTS ISO file (downloaded from ubuntu.com) as the installer disc image.
8. Enter a VM name and choose a storage location on the host disk.
9. Allocate disk space (20 GB or more recommended).
10. Under Customize Hardware, allocate at least 4 CPU cores and 2-4 GB RAM (multiple cores are needed to see real scalability differences).
11. Click Finish to create the VM.

5.3 Install Ubuntu 22.04 LTS

12. Power on the VM; it boots from the Ubuntu ISO.
13. Select "Install Ubuntu", choose language and keyboard layout.
14. Choose "Erase disk and install Ubuntu" (this affects only the virtual disk).
15. Set time zone, create a username and password, and continue.
16. Restart the VM when installation finishes and remove the installation media.
17. Log in to the new Ubuntu desktop.

5.4 Install GCC and OpenMP

18. Open a Terminal (Ctrl+Alt+T) and update the package index:

```
sudo apt update
```

19. Install build-essential, which provides GCC, G++, Make, and libgomp (OpenMP runtime):

```
sudo apt install build-essential -y
```

20. Verify GCC and OpenMP support:

```
gcc --version
echo '#include <omp.h>' | gcc -E -fopenmp -xc - > /dev/null && echo OpenMP OK
```

6. Programs and Compilation

Each program below demonstrates one synchronization approach for incrementing a shared counter (or, for the barrier program, synchronizing thread phases).

6.1 Program 1 - Race Condition (No Synchronization)

```
#include <stdio.h>
#include <omp.h>
int main(){
    long counter=0;
    #pragma omp parallel for
    for(long i=0;i<10000000;i++) counter++;
    printf("Counter=%ld\n",counter);
}
```

Compile:

```
gcc race.c -fopenmp -o race
```

6.2 Program 2 - Critical Section

```
#include <stdio.h>
#include <omp.h>
int main(){
    long counter=0;
    #pragma omp parallel for
    for(long i=0;i<10000000;i++){
        #pragma omp critical
        {counter++;}
    }
    printf("Counter=%ld\n",counter);
}
```

Compile:

```
gcc critical.c -fopenmp -o critical
```

6.3 Program 3 - Atomic Operation

```
#include <stdio.h>
#include <omp.h>
int main(){
    long counter=0;
    #pragma omp parallel for
    for(long i=0;i<10000000;i++){
        #pragma omp atomic
        counter++;
    }
    printf("Counter=%ld\n",counter);
}
```

```
```

Compile:

```
gcc atomic.c -fopenmp -o atomic
```

6.4 Program 4 - Barrier

```
#include <stdio.h>
#include <omp.h>
int main(){
    #pragma omp parallel
    {
        printf("Thread %d before barrier\n",omp_get_thread_num());
        #pragma omp barrier
        printf("Thread %d after barrier\n",omp_get_thread_num());
    }
}
```

Compile:

```
gcc barrier.c -fopenmp -o barrier
```

6.5 Program 5 - Semaphore

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
sem_t semaphore;
int counter=0;
void* work(void* a){
    for(int i=0;i<100000;i++){
        sem_wait(&semaphore);
        counter++;
        sem_post(&semaphore);
    }
    return NULL;
}
int main(){
    pthread_t t1,t2;
    sem_init(&semaphore,0,1);
    pthread_create(&t1,NULL,work,NULL);
    pthread_create(&t2,NULL,work,NULL);
    pthread_join(t1,NULL);
    pthread_join(t2,NULL);
    printf("Counter=%d\n",counter);
    sem_destroy(&semaphore);
}
```

Compile:

```
gcc semaphore.c -pthread -o semaphore
```

7. Comparison Program - Scalability & Execution Time

Program 6 below combines the None, Critical, Atomic, and Lock strategies into a single benchmark. It repeats the same 10,000,000-increment workload for each mechanism across 1, 2, 4, and 8 threads using omp_set_num_threads(), timing every run with omp_get_wtime() and checking the final counter value against the expected result. This single program is what produces the correctness/performance/scalability data used in Section 9.

```
/* compare_sync.c
 * Compares four counter-update strategies (no sync, critical section,
 * atomic operation, and a coarse-grained lock) across different thread
 * counts, to observe correctness, execution time, and scalability.
 * Compile: gcc compare_sync.c -fopenmp -o compare_sync
 * Run:     ./compare_sync
 */

#include <stdio.h>
#include <omp.h>

#define N 10000000L

long run_none(int nthreads) {
    long counter = 0;
    omp_set_num_threads(nthreads);
    #pragma omp parallel for
    for (long i = 0; i < N; i++) {
        counter++;
    }
    return counter;
}

long run_critical(int nthreads) {
    long counter = 0;
    omp_set_num_threads(nthreads);
    #pragma omp parallel for
    for (long i = 0; i < N; i++) {
        #pragma omp critical
        {
            counter++;
        }
    }
    return counter;
}

long run_atomic(int nthreads) {
    long counter = 0;
    omp_set_num_threads(nthreads);
    #pragma omp parallel for
    for (long i = 0; i < N; i++) {
        #pragma omp atomic
        counter++;
    }
    return counter;
}

long run_lock(int nthreads) {
    long counter = 0;
    omp_lock_t lock;
```

```
    omp_init_lock(&lock);
    omp_set_num_threads(nthreads);
    #pragma omp parallel for
    for (long i = 0; i < N; i++) {
        omp_set_lock(&lock);
        counter++;
        omp_unset_lock(&lock);
    }
    omp_destroy_lock(&lock);
    return counter;
}

void run_and_report(const char *name, long (*fn)(int), int nthreads) {
    double start = omp_get_wtime();
    long result = fn(nthreads);
    double elapsed = omp_get_wtime() - start;
    const char *correctness = (result == N) ? "Correct" : "INCORRECT";
    printf("%-10s | %7d | %12ld | %10s | %10.4f\n",
           name, nthreads, result, correctness, elapsed);
}

int main(void) {
    int thread_counts[] = {1, 2, 4, 8};
    int num_configs = sizeof(thread_counts) / sizeof(thread_counts[0]);

    printf("Comparing synchronization mechanisms for %ld shared increments\n\n", N);
    printf("Mechanism  | Threads | Final Count  | Correctness |   Time (s)\n");
    printf("-----------------------------------------------------------------\n");

    for (int i = 0; i < num_configs; i++) {
        int t = thread_counts[i];
        run_and_report("None",     run_none,     t);
        run_and_report("Critical", run_critical, t);
        run_and_report("Atomic",   run_atomic,   t);
        run_and_report("Lock",     run_lock,     t);
        printf("-----------------------------------------------------------------\n");
    }

    printf("\nExpected correct final count = %ld\n", N);
    printf("Observation: 'None' is fastest but incorrect (race condition).\n");
    printf("'Critical' is correct but slowest (coarse-grained locking).\n");
    printf("'Atomic' and 'Lock' are correct; 'Atomic' scales better since it\n");
    printf("uses lightweight hardware instructions instead of OS-level locks.\n");
    return 0;
}
```

Compile:

```
gcc compare_sync.c -fopenmp -o compare_sync
```

Run:

```
./compare_sync
```

8. Execution Steps (Terminal)

Step	Command	Purpose
1	sudo apt update && sudo apt install build-essential -y	Install GCC, Make, and the OpenMP runtime
2	nano race.c (repeat for each .c file)	Create/paste each program's source code
3	gcc race.c -fopenmp -o race	Compile the race-condition program
4	gcc critical.c -fopenmp -o critical	Compile the critical-section program
5	gcc atomic.c -fopenmp -o atomic	Compile the atomic-operation program
6	gcc barrier.c -fopenmp -o barrier	Compile the barrier program
7	gcc semaphore.c -pthread -o semaphore	Compile the semaphore program
8	gcc compare_sync.c -fopenmp -o compare_sync	Compile the comparison/benchmark program
9	OMP_NUM_THREADS=4 ./atomic	Run any program with a chosen thread count
10	./compare_sync	Run the full comparison across 1/2/4/8 threads and record timings

You can also set the thread count as an environment variable before running any OpenMP program:

```
export OMP_NUM_THREADS=8
./critical
```

9. Sample Output of the Comparison Program

A representative run (values vary by machine/VM and core count) is shown below:

```
Comparing synchronization mechanisms for 10000000 shared increments

Mechanism  | Threads | Final Count  | Correctness |   Time (s)
---------------------------------------------------------------
None       |       1 |     10000000 |    Correct |     0.0486
Critical   |       1 |     10000000 |    Correct |     0.1571
Atomic     |       1 |     10000000 |    Correct |     0.0567
Lock       |       1 |     10000000 |    Correct |     0.1586
---------------------------------------------------------------
None       |       2 |     10000000 |    Correct |     0.0249
Critical   |       2 |     10000000 |    Correct |     0.1584
Atomic     |       2 |     10000000 |    Correct |     0.0574
Lock       |       2 |     10000000 |    Correct |     0.1590
---------------------------------------------------------------
None       |       4 |      7545062 |  INCORRECT |     0.0250
Critical   |       4 |     10000000 |    Correct |     0.1583
Atomic     |       4 |     10000000 |    Correct |     0.0585
Lock       |       4 |     10000000 |    Correct |     0.1615
---------------------------------------------------------------
None       |       8 |      4394535 |  INCORRECT |     0.0250
Critical   |       8 |     10000000 |    Correct |     0.1579
Atomic     |       8 |     10000000 |    Correct |     0.0565
Lock       |       8 |     10000000 |    Correct |     0.1603
---------------------------------------------------------------
```

```
Expected correct final count = 10000000
Observation: 'None' is fastest but incorrect (race condition).
'Critical' is correct but slowest (coarse-grained locking).
'Atomic' and 'Lock' are correct; 'Atomic' scales better since it
uses lightweight hardware instructions instead of OS-level locks.
```

10. Observation / Comparison Table

Mechanism	Correctness	Relative Performance	Scalability
None (race condition)	**Incorrect**	Fastest (no overhead)	Poor - wrong result gets worse with more threads
Critical section	**Correct**	Slowest (serializes all threads)	Poor - contention grows with thread count
Atomic operation	**Correct**	Fast (hardware-level)	Good - low overhead even as threads increase
Lock (omp_lock_t)	**Correct**	Slow (similar to critical)	Poor to Moderate
Barrier	**Synchronization only**	Waiting overhead	Moderate - limited by slowest thread
Semaphore	**Correct**	Moderate (system-call overhead)	Moderate

11. Conclusion

Atomic operations provide the best balance of correctness and performance for simple shared-variable updates because they rely on lightweight hardware instructions. Critical sections and locks guarantee correctness but limit scalability since threads must wait in turn to enter the protected region. Barriers do not protect data but synchronize the phases of a parallel computation, at the cost of waiting for the slowest thread. Semaphores generalize mutual exclusion (and can allow a controlled number of threads into a region) at a moderate overhead. The comparison benchmark confirms that unsynchronized access is fastest but unreliable, while the choice among critical/atomic/lock/semaphore is a trade-off between strength of guarantee, overhead, and scalability as thread count grows.